

server2 immutable raw records

```
$ cargo run --release --example raw_records_smoke -- http://127.0.0.1:PORT
PUT Created; repeat verified; raw_sha256=7541bba8f81568e382184d838176c9ce0883f2c4234cac53c2b833da10f336c9 bytes=4
PUT Created; repeat verified; raw_sha256=957650d325cbb9d224ccea58a90cc112cad7d3c029da3ced45734b31ed8627ed bytes=15
PUT Created; repeat verified; raw_sha256=957650d318e41992856dc73bfedbb122ad8fa0b5343cd1f64e3083bf988e4e33 bytes=15
RESTORE verified 7541bba8f81568e382184d838176c9ce0883f2c4234cac53c2b833da10f336c9
RESTORE verified 957650d325cbb9d224ccea58a90cc112cad7d3c029da3ced45734b31ed8627ed
RESTORE verified 957650d318e41992856dc73bfedbb122ad8fa0b5343cd1f64e3083bf988e4e33
PASS: exact binary, both collision bodies, conditional repeat and empty-client restore; no catalog/client publication
```

S05 implements an internal Rust record store. No weather/geocode route is switched. A completed body PUT acknowledges that body only; S06 must publish all identity catalogs and the complete fetch group before any new client weather success.

Local verification

From server2/, run `cargo test --workspace` for codec and loopback HTTP scenarios. Run `cargo fmt --check`, `cargo clippy --workspace --all-targets -- -D warnings`, and `cargo build --release --examples`. No Mongo, provider keys or live AWS are used. The tests own ephemeral loopback peers and dummy credentials, with no network relay.

For an independent S3-compatible local peer configured with bucket `server2-local` and dummy credentials `server2-local / server2-local-secret`, run:

```
cargo run --release --example raw_records_smoke -- http://127.0.0.1:PORT
```

The example accepts only a literal loopback HTTP endpoint. It publishes exact binary EUC-KR-like bytes and both prefix-collision bodies, verifies repeated conditional PUTs, then constructs an empty client and restores all three records. It prints only frozen fixture hashes and lengths. The selected actual output is from the independent S03 local peer, not a real AWS measurement. See the PDF manual.

For the controlled delayed-commit HTTP smoke, the same release example supports `--uncertainty-smoke` after the endpoint. This requires a scripted loopback peer, not the ordinary S03 peer: first PUT accepts but loses its response, HEAD returns 404, retry rejects with 401/403/400/202, and POST `/__test/commit` releases the first commit after publication returns Ambiguous. Exact GET then verifies the saved body; a separate direct-first403 writes no object and stays Status(403). The deterministic automated wire regression controls the same commit ordering without sleeps. No S06 catalog or client publication is acknowledged.

Format and downstream integration

`RawRecord::new` accepts a typed provider identity and exact `Arc<[u8]>` bytes. Grid keys carry KMA grid IDs. World keys carry the existing 0.02-degree cell centres in integer hundredths. Named/station keys are bounded ASCII identities; a hash or generic name does not authorize storing precise user coordinates. Provider adapters remain responsible for coordinate reduction and for rejecting quota/auth/XML/error bodies before storage. Non-UTF8 bodies remain unmodified.

The object key is:

```
raw/v2/{source}/{kind}/{YYYY}/{MM}/{DD}/{slot}/{key_sha256}/
{fetched_at_ms}-{raw_sha256}.raw.gz
```

The explicit placement date is the provider-local first date covered, including Visual Crossing ranges, not the fetch UTC date. Source/kind/slot cannot contain slashes. Typed keys have canonical tagged JSON; both identity hashes use full SHA-256. Every distinct fetch revision stays separate.

Gzip is one deterministic member (mtime 0, OS 255, compression level 3) containing only provider bytes. Content-Type is `application/gzip`; Content-Encoding is absent. `x-amz-meta-s2-record` contains base64 canonical JSON for schema 2, full identity, provider status/content type/raw length, optional ordered-page metadata and an optional identity-only fetch-group reference. `x-amz-meta-s2-gzip-sha256` is the compressed body's full hash. Total user metadata is limited to 2 KiB including base64 expansion. A group reference has full group/member/partition digests and a bounded member count; its descriptor key is `index/v2/groups/{group_sha256}.json`. with `_fetch_group` validates and attaches the reference before PUT. S05 accepts one group reference per immutable body and validates its shape only. S06 must resolve the descriptor to the full ordered

partition/member list and expected page counts, require group membership for grouped acquisitions, and reject missing or inconsistent descriptors. It must decide canonical shared-body ownership before publishing; S05 does not implement multiple group affiliations or descriptor repair. S06 supplies this reference before PUT and validates complete membership. An ungrouped body does not prove a complete paginated or cross-partition acquisition.

Failures and limits

PUT always sends If-None-Match: * and Content-MD5 of compressed bytes. Only HTTP 200 acknowledges a completed PUT; 201/202/204 do not. On 412, 409, 5xx or an ambiguous transport outcome, HEAD must match the canonical envelope and configured size limit. GET must agree with that stored HEAD and verify its own compressed hash, full raw identity, envelope, length and gzip integrity. A valid alternative gzip encoding of the same exact raw bytes/envelope is accepted across compressor builds; the new writer's gzip length/hash need not equal the stored encoding. HEAD alone and ETag never establish the raw identity. If HEAD finds no object, one retry uses the identical conditional key and bytes; there is no overwrite, new fetch timestamp or hidden retry. A missing HEAD does not rule out a delayed first PUT commit. Once any PUT outcome is unknown, later rejection/local failure also returns Ambiguous, unless exact identity verification resolves it. A first PUT directly rejected with 403 keeps Status(403); Corrupt/Invalid remain fail-closed. There are still at most two PUTs within the original deadline.

Default limits are 8 MiB raw, 8 MiB + 64 KiB compressed, 16 I/O operations, 2 CPU workers and a 3-second monotonic operation deadline. I/O admission fails closed with Capacity before a PUT is attempted. Admitted compression/decode waits for CPU within the same deadline. A separate pool of io download-buffer slots spans GET, CPU wait and decode, and stays owned by a running decoder even after caller cancellation. Network GET therefore uses up to 16 slots independently of the two CPU workers. Retained compressed payloads are bounded by $\text{io} \times \text{gzip_bytes}$ (129 MiB by default), plus decoded worker payloads of at most $\text{cpu} \times \text{raw_bytes}$ (16 MiB); returned records, upload buffers, allocation capacity and networking overhead require separate caller/RSS accounting. This is a logical payload bound, not a process RSS promise. Waiters do not use try-acquire CPU rejection, and their queue is limited by I/O admission. The 16-load/50 ms peer regression checks concurrent downloads and records observed timing; it is not an AWS latency benchmark. A pre-PUT deadline returns Timeout. Once PUT has been sent, a verification deadline or transport uncertainty returns Ambiguous, never Capacity: reconcile the same identity, do not infer not-written or fetch a new provider response. Reconciliation integrity failures stay Corrupt/Invalid; every other HEAD/GET failure, including 403/503, becomes Ambiguous. Direct definitive PUT rejections keep their status only when no earlier PUT is unresolved, and are distinct from uncertain reconciliation. Compression is capped while writing; downloads are capped while reading, even with false Content-Length. Decode validates CRC, full raw hash, length, exactly one gzip member and no trailing bytes. CPU permits stay inside blocking closures if the caller cancels. Limits are per store/operation, not a hard global RSS guarantee: the caller must bound admitted bytes and shared work.

HttpS3Transport signs requests with supplied credentials using `rusty-s3` and sends with `request`, without proxies, redirects, automatic retries or decompression. HTTP is permitted only for literal loopback; other endpoints require TLS. Use a cloned `RefreshableCredentials` handle with `HttpS3Transport::with_credentials` and use `handle.replace(credentials).await` before supplied credentials expire. This keeps the same transport, connection pool and store admission limits. Each PUT/HEAD/GET signs from one coherent snapshot; in-flight requests retain their snapshot. Credential snapshots and replacement use a fair asynchronous lock, yield while contended, and have a 3-second wait ceiling (Timeout on expiry). `RawRecordStore`'s absolute operation deadline also bounds that wait; no async worker is blocked by a synchronous lock. The lock guard never crosses signing/network work. The caller owns expiry, refresh scheduling and instance-role acquisition; no AWS/environment discovery is added. Fixed credentials through new remain useful for local tests. Signed URLs and credentials are never returned in errors or logged. Public synchronous codec helpers require the caller to schedule CPU work; `RawRecordStore` schedules and bounds that work.

An interrupted caller can leave one immutable body orphan; the store does not claim catalog publication or background durability drain. S06 discovers and repairs complete groups. Never delete raw keys, old versions or create delete markers: conditional S3 writes check the current version, so IAM/lifecycle no-delete policy is also required. S3 outage cannot authorize a new undurable client success.

The local peers validate the protocol subset and dummy credential/signature presence, not AWS SigV4 cryptography, IAM or tail latency. Production credentials, staging verification and S09 benchmarks remain separate gates. No filesystem store, normalized weather copy, shared-instance cache or provider call is implemented.

Primary contracts: [AWS PutObject](#), [conditional writes](#), [metadata limits](#), [rusty-s3](#), and [request](#).